

Emulation and debugging techniques

Debugging techniques

The fundamental aim of a debugging methodology is to restrict the introduction of untested software or hardware to a single item. This is good practice and of benefit to the design and implementation of any system, even those that use emulation early on in the design cycle.

It is to prevent this integration of two unknowns that simulation programs to simulate and test software, hardware or both can play a critical part in the development process.

High level language simulation

If software is written in a high level language, it is possible to test large parts of it without the need for the hardware at all. Software that does not need or use I/O or other system dependent facilities can be run and tested on other machines, such as a PC or a engineering workstation. The advantage of this is that it allows a parallel development of the hardware and software and added confidence, when the two parts are integrated, that it will work.

Using this technique, it is possible to simulate I/O using the keyboard as input or another task passing input data to the rest of the modules. Another technique is to use a data table which contains data sequences that are used to test the software.

This method is not without its restrictions. The most common mistake with this method is the use of non-standard libraries which are not supported by the target system compiler or environment. If these libraries are used as part of the code that will be transferred, as opposed to providing a user interface or debugging facility, then the modifications needed to port the code will devalue the benefit of the simulation.

The ideal is when the simulation system is using the same library interface as the target. This can be achieved by using the target system or operating system as the simulation system or using the same set of system calls. Many operating systems support or provide a UNIX compatible library which allows UNIX software to be ported using a simple recompilation. As a result, UNIX systems are often employed in this simulation role. This is an advantage which the POSIX compliant operating system Lynx offers.

This simulation allows logical testing of the software but rarely offers quantitative information unless the simulation

environment is very close to that of the target, in terms of hardware and software environments.

Low level simulation

Using another system to simulate parts of the code is all well and good, but what about low level code such as initialisation routines? There are simulation tools available for these routines as well. CPU simulators can simulate a processor, memory system and, in some cases, some peripherals and allow low level assembler code and small HLL programs to be tested without the need for the actual hardware. These tools tend to fall into two categories: the first simulate the programming model and memory system and offer simple debugging tools similar to those found with an onboard debugger. These are inevitably slow, when compared to the real thing, and do not provide timing information or permit different memory configurations to be tested. However, they are very cheap and easy to use and can provide a low cost test bed for individuals within a large software team. There are even shareware simulators for the most common processors such as the one from the University of North Carolina which simulates an MC68000 processor.

```
<D0> =00000000 <D4> =00000000 <A0> =00000000 <A4> =00000000
<D1> =00000000 <D5> =0000abcd <A1> =00000000 <A5> =00000000
<D2> =00000000 <D6> =00000000 <A2> =00000000 <A6> =00000000
<D3> =00000000 <D7> =00000000 <A3> =00000000 <A7> =00000000
trace: on      sstep: on      cycles: 416 <A7'>= 00000f00
      cn tr st rc      T S INT XNZVC <PC> = 00000090
port1 00 00 82 00 SR = 1010101111011111
-----
executing a ANDI      instruction at location 58
executing a ANDI      instruction at location 5e
executing a ANDI      instruction at location 62
executing a ANDI_TO_CCR instruction at location 68
executing a ANDI_TO_SR instruction at location 6c
executing a OR         instruction at location 70
executing a OR         instruction at location 72
executing a OR         instruction at location 76
executing a ORI        instruction at location 78
executing a ORI        instruction at location 7e
executing a ORI        instruction at location 82
executing a ORI_TO_CCR instruction at location 88
executing a ORI_TO_SR  instruction at location 8c
TRACE exception occurred at location 8c.
Execution halted
```

Example display from the University of North Carolina 68k simulator

The second category extends the simulation to provide timing information based on the number of clock cycles. Some simulators can even provide information on cache performance, memory usage and so on, which is useful data for making hardware decisions. Different performance memory systems can be exercised using the simulator to provide performance data. This type of information is virtually impossible to obtain

without using such tools. These more powerful simulators often require very powerful hosts with large amounts of memory. SDS provide a suite of such tools that can simulate a processor and memory and with some of the integrated processors that are available, even emulate onboard peripherals such as LCD controllers and parallel ports.

Simulation tools are becoming more and more important in providing early experience of and data about a system before the hardware is available. They can be a little impractical due to their performance limitations — one second of processing with a 25 MHz RISC processor taking 2 hours of simulation time was not uncommon a few years ago — but as workstation performance improves, the simulation speed increases. With instruction level simulators it is possible with a top of the range workstation to get simulation speeds of 1 to 2 MHz.

Onboard debugger

The onboard debugger provides a very low level method of debugging software. Usually supplied as a set of EPROMs which are plugged into the board or as a set of software routines that are combined with the applications code, they use a serial connection to communicate with a PC or workstation. They provide several functions: the first is to provide initialisation code for the processor and/or the board which will normally initialise the hardware and allow it to come up into a known state. The second is to supply basic debugging facilities and, in some cases, allow simple access to the board's peripherals. Often included in these facilities is the ability to download code using a serial port or from a floppy disk.

>TR

```
PC=000404 SR=2000 SS=00A00000 US=00000000      X=0
A0=00000000 A1=000004AA A2=00000000 A3=00000000 N=0
A4=00000000 A5=00000000 A6=00000000 A7=00A00000 Z=0
D0=00000001 D1=00000013 D2=00000000 D3=00000000 V=0
D4=00000000 D5=00000000 D6=00000000 D7=00000000 C=0
----->LEA      $000004AA,A1
```

>TR

```
PC=00040A SR=2000 SS=00A00000 US=00000000      X=0
A0=00000000 A1=000004AA A2=00000000 A3=00000000 N=0
A4=00000000 A5=00000000 A6=00000000 A7=00A00000 Z=0
D0=00000001 D1=00000013 D2=00000000 D3=00000000 V=0
D4=00000000 D5=00000000 D6=00000000 D7=00000000 C=0
----->MOVEQ    #19,D1
```

>

Example display from an onboard M68000 debugger

When the board is powered up, the processor fetches its reset vector from the table stored in EPROM and then starts to

initialise the board. The vector table is normally transferred from EPROM into a RAM area to allow it to be modified, if needed. This can be done through hardware, where the EPROM memory address is temporarily altered to be at the correct location for power-on, but is moved elsewhere after the vector table has been copied. Typically, a counter is used to determine a preset number of memory accesses, after which it is assumed that the table has been transferred by the debugger and the EPROM address can safely be changed.

The second method, which relies on processor support, allows the vector table to be moved elsewhere in the memory map. With the later M68000 processors, this can also be done by changing the vector base register which is part of the supervisor programming model.

The debugger usually operates at a very low level and allows basic memory and processor register display and change, setting RAM-based breakpoints and so on. This is normally performed using hexadecimal notation, although some debuggers can provide a simple disassembler function. To get the best out of these systems, it is important that a symbol table is generated when compiling or linking software, which will provide a cross-reference between labels and symbol names and their physical address in memory. In addition, an assembler source listing which shows the assembler code generated for each line of C or other high level language code is invaluable. Without this information it can be very difficult to use the debugger easily. Having said that, it is quite frustrating having to look up references in very large tables and this highlights one of the restrictions with this type of debugger.

While considered very low level and somewhat limited in their use, onboard debuggers are extremely useful in giving confidence that the board is working correctly and working on an embedded system where an emulator may be impractical. However, this ability to access only at a low level can also place severe limitations on what can be debugged.

The first problem concerns the initialisation routines and in particular the processor's vector table. Breakpoints use either a special breakpoint instruction or an illegal instruction to generate a processor exception when the instruction is executed. Program control is then transferred to the debugger which displays the breakpoint and associated information. Similarly, the debugger may use other vectors to drive the serial port that is connected to the terminal.

This vector table may be overwritten by the initialisation routines of the operating system which can replace them with its own set of vectors. The breakpoint can still be set but when it is reached, the operating system will see it instead of the debugger and not pass control back to it. The system will normally crash because it is not expecting to see a breakpoint or an illegal instruction!

To get around this problem, the operating system may need to be either patched so that its initialisation routine writes the debugger vector into the appropriate location or this must be done using the debugger itself. The operating system is single stepped through its initialisation routine and the instruction that overwrites the vector simply skipped over, thus preserving the debugger's vector. Some operating systems can be configured to preserve the debugger's exception vectors, which removes the need to use the debugger to preserve them.

A second issue is that of memory management where there can be a problem with the address translation. Breakpoints will still work but the addresses returned by the debugger will be physical, while those generated by the symbol table will normally be logical. As a result, it can be very difficult to reconcile the physical address information with the logical information.

The onboard debugger provides a simple but sometimes essential way of debugging VMEbus software. For small amounts of code, it is quite capable of providing a method of debugging which is effective, albeit not as efficient as a full blown symbolic level debugger — or as complex or expensive. It is often the only way of finding out about a system which has hung or crashed.

Task level debugging

In many cases, the use of a low level debugger is not very efficient compared with the type of control that may be needed. A low level debugger is fine for setting a breakpoint at the start of a routine but it cannot set them for particular task functions and operations. It is possible to set a breakpoint at the start of the routine that sends a message, but if only a particular message is required, the low level approach will need manual inspection of all messages to isolate the one that is needed — an often daunting and impractical approach!

To solve this problem, most operating systems provide a task level debugger which works at the operating system level. Breakpoints can be set on system circumstances, such as events, messages, interrupt routines and so on, as well as the more normal memory address. In addition, the ability to filter messages and events is often included. Data on the current executing tasks is provided, such as memory usage, current status and a snapshot of the registers.

Symbolic debug

The ability to use high level language instructions, functions and variables instead of the more normal addresses and their contents is known as symbolic debugging. Instead of using an assembler listing to determine the address of the first instruction of a C function and using this to set a breakpoint,